

‘Big Tech’ Stock Predictor - Final Report

Griffin Kuchar

May 8th, 2026

1 Abstract

The US technology industry includes some of the most fascinating and well-performing stocks of the last few years. Using the previous six years of each stock’s OHLCV and technical indicators as data, this project trained and evaluated various temporal machine learning models on their ability to correctly predict which one of the eight most popular technology stocks will perform best the next day.

Source code: <https://github.com/gkuchar/big-tech-stock-classifier>

2 Problem Statement & Motivation

This project aims to solve the complex, yet highly sought after problem of discovering hidden, underlying patterns in market data to gain a financial edge in stock trading. The importance of this project derives from its coupling with the high-implications finance domain, and the massive profits that can arise from even an insignificant edge on the rest of traders. The challenge and difficulty of this classification project is due to the inherent unpredictability of the stock market.

3 Dataset Description & Feature Engineering

The models were trained and evaluated on the six year history (May 7th, 2020 through May 7th, 2026) across eight different US technology stocks: Apple (AAPL), AMD (AMD), Amazon (AMZN), Google (GOOGL), Meta (META), Microsoft (MSFT), Nvidia (NVDA), and Tesla (TSLA). Each row of data was engineered to the combined 1 day, 5 day, and 21 day percent changes of closing price, augmented with technical indicators of each of the eight stocks. The raw market (OHLCV) data was sourced from the yahoo finance (yfinance) API, and the technical indicator data was computed using the Pandas Technical Analysis (pandas.ta) library. Each row’s label consisted of the best performing stock by percent change for the following day. Preprocessing included collating the

desired features into a singular frame and dropping all empty rows. Train/test splitting and standard scaling was implemented dynamically at fitting time due to the time series nature of the data utilizing an expanding window optimization methodology. In total, the finalized dataset consisted of 717 rows with 120 feature columns.

4 Methodology

For this project, three baselines were established:

1. **Random:** Predicting one of the eight stocks at random. This is the purest baseline to compare with.
2. **Momentum:** Predicting the stock that performed the best the previous day. Encapsulates the idea that well-performing stocks are likely to continue to perform well.
3. **Reversion:** Predicting the stock that performed the worst the previous day. Carries out the theory that in the long term, stocks naturally drift towards their average; assumes a stock has a high potential to ‘bounce back’.

This project tested three additional models against the baselines:

1. **Softmax Regression:** A multi-class logistic regression, incorporating L2 regularization.
2. **XGBoost:** Decision-tree-based model that augments boosting between new tree iterations. Specific hyperparameters included: 50 independent trees, a max depth of three splits per tree, and log loss optimization.
3. **MLP Neural Network:** A two layer neural network, one hidden layer with 50 neurons, and one output layer of eight possible classification predictions. Hyperparameters included a ReLU activation function, L2 regularization with a strength of .1, a constant learning rate (.001), and a maximum number of 200 epochs.

Each model was trained using an expanding window time series approach. Each iteration of the expanding window training and evaluation used days 1 to $n - 1$ as the model’s train set, and day n as its test set. n is initialized to the modifiable developer parameter `WINDOW_STARTING_SIZE`, and increments until all of the days in the dataset have been trained and evaluated on. The `WINDOW_STARTING_SIZE` of this evaluation was chosen to be 252: one full year of market trading days.

Standard scaling was fit on the new training set, and then transformed on both the new train and the new test set on every iteration. This design decision was implemented to prevent data leakage. Since standard scaling requires

computing the mean and standard deviation of the set, if fit on the entire set, the test set could cheat the evaluation as it would know the mean and standard deviation of the set when it makes its prediction. The typical solution to preventing standard scaling data leakage is to fit on the train set once, and then transform both the train and test sets. However, since the train set changed for every expanding window iteration, refitting on each iteration is also required.

Softmax Regression was chosen to evaluate a linear model, XGBoost was chosen to evaluate a tree-based model that specialized in complex data, and an MLP NN was chosen to evaluate a non-linear model.

5 Evaluation

Due to the multi-classification structure of this project, the primary evaluation metric was accuracy. This was chosen because of how well it corresponds to the problem this project aims to solve: predicting, as accurately as possible, the next day's best performing stock. The confusion matrix of each model was evaluated alongside accuracy for the same reasoning. Additionally, the weighted F1 score of each model was used as an evaluation metric. Weighted F1 score allows the capturing of precision and recall in a multi-classification model. The weighted aspect is important in this dataset as some labels are heavily more present than others (up to 4x more present).

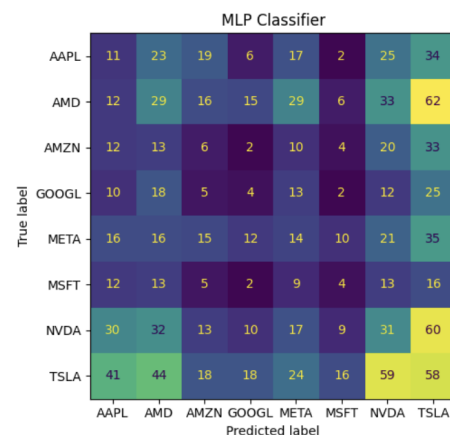
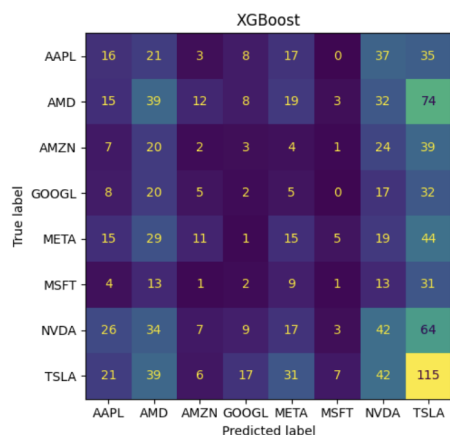
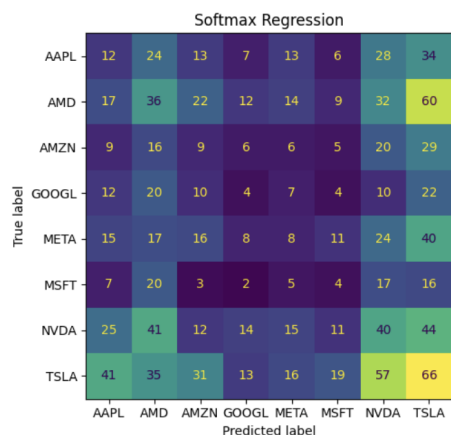
As previously mentioned in section 4 with more detail, expanding window time series validation was implemented.

6 Results and Deliverables

6.1 Quantitative Metrics

	Accuracy	Weighted F1
Softmax Regression	0.1466	0.1427
XGBoost	0.1900	0.1703
MLP Classifier	0.1286	0.1262
--- Baselines ---	NaN	NaN
Random	0.1250	NaN
Momentum	0.1629	NaN
Reversion	0.1559	NaN

6.2 Confusion Matrices



6.3 Analysis

In terms of both accuracy and F1 Score, XGBoost is the best performing model, exhibiting a 52% increase in accuracy over the random baseline. This is likely due to its ability to pick up on complex patterns in the data in conjunction with its ability to improve the tree structure over each iteration. The MLP struggled the most. This could be due to the small data set size resulting in model overfitting. The confusion matrices reveal a model bias toward AMD and TSLA, however this is justified by the imbalance in the dataset favoring these two stocks.

The models may have been limited by the small dataset size of only six years of market data. This issue naturally could be overcome by expanding the window to years greater than six years, however due to the recent explosion of technology stocks in the last few years, I could foresee expanding the window adding more confusion into the data instead. This is a hypothesis to test in the future, in addition to experimenting with different models and/or different hyperparameters. Testing a RNN instead of a MLP NN would be an interesting experiment as RNNs are able to handle time series data far better than a MLP NN because of their ability to retain a ‘memory’ of previous inputs.